

**Does Preprocessing Matter? A Python-Based Comparison of Missing-Data Strategies and
Classifiers for Early Diabetes Prediction**

[Student Name]

[Department, University]

[Course Number]: [Course Name]

[Instructor Name]

[Due Date]

Abstract

Diabetes affects more than half a billion adults, and many cases remain undiagnosed until complications appear. Machine-learning screening models are often proposed as a remedy, yet studies rarely report how data-preparation choices, especially the handling of physiologically impossible zero values, shape performance. This paper asks whether preprocessing matters as much as model choice in Python-based diabetes prediction. Using the Pima Indians Diabetes dataset ($N = 768$; 34.9% positive), I built leakage-safe scikit-learn pipelines crossing four missing-data strategies (zeros retained, median, k-nearest-neighbor, and iterative imputation) with three classifiers (logistic regression, random forest, and XGBoost). All twelve configurations were evaluated with five-times-repeated 10-fold cross-validation, run in parallel with Dask, and then tested on a stratified hold-out set. Mean cross-validated ROC-AUC ranged only from .823 to .838. Treating zeros as missing improved AUC for every model, but by less than .01, far below the fold-to-fold standard deviation (about .05). On the hold-out set, logistic regression matched the ensembles (AUC = .823, recall = .701) while remaining interpretable, with glucose and body-mass index dominating its predictions. A subgroup audit, however, showed that sensitivity fell from .822 in patients aged 30 or older to .455 in those under 30, a gap that aggregate metrics conceal. For small tabular clinical datasets, analysts should prioritize sound pipeline design, calibration, and subgroup evaluation over algorithmic complexity, and the open-source Python stack makes such rigor inexpensive.

Keywords: Python, diabetes prediction, missing-data imputation, scikit-learn, algorithmic fairness

Does Preprocessing Matter? A Python-Based Comparison of Missing-Data Strategies and Classifiers for Early Diabetes Prediction

Introduction

Diabetes mellitus is one of the fastest-growing chronic diseases in the world. The International Diabetes Federation estimated that 537 million adults were living with diabetes in 2021, projected to reach 783 million by 2045, and that almost half were undiagnosed (Sun et al., 2022). Because type 2 diabetes progresses silently, early identification of high-risk individuals allows intervention before irreversible organ damage. Health systems therefore need screening tools that are cheap, scalable, and easy to audit.

Machine learning (ML) has been widely promoted as such a tool (Rajkomar et al., 2019; Topol, 2019), and diabetes is one of the most studied conditions (Kavakiotis et al., 2017). Much of this work is carried out in Python, whose stack of NumPy (Harris et al., 2020), pandas (McKinney, 2010), scikit-learn (Pedregosa et al., 2011), XGBoost (Chen & Guestrin, 2016), and Dask (Rocklin, 2015) takes an analyst from raw records to a validated model in a few hundred lines of code. Yet clinical data routinely contain missing or implausible values, and how they are cleaned can change results, sometimes through data leakage that inflates reported accuracy (Kapoor & Narayanan, 2023).

The problem addressed here is therefore not whether ML *can* predict diabetes, which is well established, but how much preprocessing matters relative to algorithm choice. The research question is: *How do missing-data handling strategies and classifier choice, implemented in leakage-safe Python pipelines, jointly affect the accuracy, calibration, interpretability, and subgroup fairness of diabetes risk prediction?*

The thesis of this paper is that, on small tabular clinical data, preprocessing and model choice produce only modest differences in discrimination, so the real value of the Python ecosystem lies less in sophisticated algorithms than in the ease with which it supports rigorous evaluation: leakage-proof pipelines, repeated cross-validation that exposes uncertainty, interpretable models, and subgroup audits that reveal inequities hidden by aggregate scores.

Literature Review

Data Analytics and Machine Learning in Healthcare

Major reviews of clinical ML (Rajkomar et al., 2019; Rajpurkar et al., 2022; Topol, 2019) note that impressive retrospective accuracy has rarely translated into clinical benefit, partly because development studies are poorly reported and insufficiently validated. The TRIPOD statement (Collins et al., 2015) standardizes how prediction models should be reported, including the handling of missing data, and Van Calster et al. (2019) call calibration the "Achilles heel" of predictive analytics because it is so often ignored.

Machine-Learning Models for Diabetes Prediction

The Pima Indians Diabetes dataset, first used for ML by Smith et al. (1988), who reported roughly 76% accuracy, has become a standard benchmark. Kavakiotis et al. (2017) found support vector machines, decision trees, and ensembles to be the most popular methods in diabetes research but noted wide heterogeneity in evaluation designs. Zou et al. (2018) found that random forests performed best on hospital and Pima data and that feature selection had little effect.

A counterweight to this literature is the systematic review by Christodoulou et al. (2019), which found no evidence that ML outperformed logistic regression across 71 clinical prediction studies once comparisons at high risk of bias were excluded. Rudin (2019) argues that when an

interpretable model performs as well as a black box, high-stakes domains should prefer it over post-hoc explanations.

Missing Data and Preprocessing

Clinical records are rarely complete, and Sterne et al. (2009) warn that both complete-case analysis and naive single imputation can bias results. Multivariate imputation by chained equations (MICE; van Buuren & Groothuis-Oudshoorn, 2011) and k-nearest-neighbor (KNN) imputation (Troyanskaya et al., 2001) are established alternatives, and scikit-learn implements median, KNN, and MICE-style imputers within a Pipeline interface that fits each step only on training folds (Pedregosa et al., 2011). This matters because fitting imputers or scalers on the full dataset before splitting is a textbook form of leakage (Kaufman et al., 2012), a leading cause of irreproducible ML-based science (Kapoor & Narayanan, 2023).

Fairness and Explainability

Obermeyer et al. (2019) showed that a widely used commercial algorithm underestimated the health needs of Black patients because it used cost as a proxy for need. Rajkomar et al. (2018) argue that fairness must be designed into clinical ML from the start, and Mehrabi et al. (2021) catalog bias metrics such as group-wise true- and false-positive rates, in the spirit of equal opportunity (Hardt et al., 2016). For explanation, permutation importance (Fisher et al., 2019) and Shapley values (Lundberg & Lee, 2017) are widely used, though Rudin (2019) cautions that they only approximate the model.

Identified Gaps

Three gaps emerge. First, diabetes-prediction studies seldom compare missing-data strategies systematically, even though the Pima data's zeros-as-values flaw is well known. Second, many studies report a single train-test split without variability or calibration, so small

differences are over-interpreted. Third, subgroup performance is rarely reported. This study addresses all three using only open-source Python tools.

Methodology

Research Design and Data

The study uses a quantitative, comparative experimental design on the public Pima Indians Diabetes dataset (Smith et al., 1988): 768 women aged 21 or older of Pima heritage, eight predictors (pregnancies, two-hour plasma glucose, diastolic blood pressure, triceps skinfold, two-hour serum insulin, body-mass index [BMI], a diabetes pedigree function summarizing family history, and age), and a binary outcome indicating diabetes within five years. The data are de-identified and public. As Table 1 shows, several variables contain physiologically impossible zeros; nearly half of insulin values and almost 30% of skinfold values are zero, which in practice means "not measured."

Table 1

Descriptive Statistics of Predictors (N = 768)

Variable	<i>M</i>	<i>SD</i>	Min	Max	% zeros
Pregnancies	3.85	3.37	0.0	17.0	14.4
Glucose (mg/dL)	120.89	31.97	0.0	199.0	0.7
Blood pressure (mm Hg)	69.11	19.36	0.0	122.0	4.6
Skinfold (mm)	20.54	15.95	0.0	99.0	29.6
Insulin (μ U/mL)	79.80	115.24	0.0	846.0	48.7
BMI (kg/m^2)	31.99	7.88	0.0	67.1	1.4
Diabetes pedigree	0.47	0.33	0.08	2.42	0.0
Age (years)	33.24	11.76	21.0	81.0	0.0

Note. Zeros in pregnancies are valid; zeros in glucose, blood pressure, skinfold, insulin, and BMI represent missing measurements. Outcome prevalence = 34.9% (268 of 768).

Tools and Pipeline

Analysis used Python 3.11 with pandas, NumPy, scikit-learn 1.9, XGBoost 3.2, Dask, Matplotlib, and Seaborn (Harris et al., 2020; Hunter, 2007; McKinney, 2010; Waskom, 2021).

Each configuration was a scikit-learn Pipeline, so all preprocessing was learned only from training folds (Kaufman et al., 2012). Four preprocessing strategies were compared: (a) *raw* (zeros retained, standardization only); (b) *median imputation*; (c) *KNN imputation* with five neighbors (Troyanskaya et al., 2001); and (d) *iterative*, MICE-style *imputation* (van Buuren & Groothuis-Oudshoorn, 2011). Each was paired with L2-regularized logistic regression, a 300-tree random forest (Breiman, 2001), and a shallow XGBoost model (300 trees, depth 3, learning rate 0.05). Class imbalance was handled with balanced class weights. Hyperparameters were fixed in advance, so differences reflect methods rather than tuning effort.

Evaluation

Each configuration was evaluated with stratified 10-fold cross-validation repeated five times (600 fits in total). The twelve independent experiments ran in parallel as Dask delayed tasks in under four minutes. Metrics were ROC-AUC, accuracy, precision, recall (sensitivity), F1, and the Brier score, which reflects calibration (Van Calster et al., 2019). Paired fold-level AUCs were compared with Wilcoxon signed-rank tests, interpreted cautiously because overlapping training sets make such tests optimistic (Nadeau & Bengio, 2003). A stratified 75/25 split then provided a hold-out test set ($n = 192$). Interpretability was assessed with odds ratios and permutation importance (Fisher et al., 2019), and a fairness audit compared performance for patients under 30 versus 30 or older, the only demographic split available. Core code appears in the Appendix.

Limitations of the Design

The dataset is small, decades old, and drawn from one population of women, so findings may not generalize to other groups or to contemporary records, and subgroup estimates are

imprecise. Hyperparameters were not tuned, and the retrospective design offers no evidence of clinical utility.

Results and Discussion

Effect of Preprocessing and Model Choice

Table 2 summarizes the cross-validated results. The most striking finding is how *narrow* the range is: mean AUC varied only from .823 (XGBoost, raw) to .838 (random forest with median imputation; logistic regression with KNN imputation), while the fold-to-fold standard deviation was about .05 for every configuration, roughly three times the entire spread between the best and worst pipelines.

Table 2

Cross-Validated Performance by Preprocessing Strategy and Classifier

Preprocessing	Model	ROC-AUC	Recall	F1	Brier
Raw (zeros kept)	Logistic Regression	.830 (.049)	.724 (.091)	.671 (.064)	.168 (.025)
Raw (zeros kept)	Random Forest	.832 (.048)	.730 (.082)	.673 (.062)	.164 (.022)
Raw (zeros kept)	XGBoost	.823 (.047)	.726 (.085)	.673 (.061)	.172 (.027)
Median imputation	Logistic Regression	.837 (.048)	.712 (.097)	.670 (.071)	.166 (.025)
Median imputation	Random Forest	.838 (.047)	.756 (.081)	.689 (.059)	.161 (.023)
Median imputation	XGBoost	.829 (.047)	.736 (.081)	.682 (.060)	.167 (.028)
KNN imputation	Logistic Regression	.838 (.047)	.720 (.101)	.674 (.072)	.166 (.025)
KNN imputation	Random Forest	.834 (.050)	.747 (.085)	.682 (.069)	.162 (.024)
KNN imputation	XGBoost	.825 (.050)	.728 (.082)	.673 (.066)	.170 (.031)
Iterative (MICE-style)	Logistic Regression	.837 (.048)	.719 (.097)	.675 (.072)	.166 (.025)
Iterative (MICE-style)	Random Forest	.836 (.047)	.750 (.080)	.680 (.063)	.162 (.024)
Iterative (MICE-style)	XGBoost	.828 (.050)	.745 (.086)	.682 (.062)	.169 (.030)

Note. Values are means (standard deviations) over 10-fold stratified cross-validation repeated five times (50 folds). Lower Brier scores are better. Threshold = 0.5.

Converting impossible zeros to missing values improved mean AUC for all three classifiers, by .002 to .007. Gains were most consistent for the ensembles: median imputation raised random-forest AUC from .832 to .838 (Wilcoxon $p = .020$) and XGBoost AUC from .823 to .829 ($p = .004$), and lifted random-forest recall from .730 to .756. For logistic regression the

gains were indistinguishable from noise ($p > .60$). The more sophisticated KNN and iterative imputers did not beat simple median imputation, echoing Zou et al.'s (2018) finding that preprocessing refinements yield limited gains on this benchmark. *Recognizing missingness* matters more than the imputer used.

Model choice showed a similar pattern. Under median imputation, random forest and logistic regression were indistinguishable (AUC difference = .001, $p = .83$), and both slightly outperformed the untuned XGBoost model, replicating Christodoulou et al.'s (2019) conclusion that ML offers no systematic advantage over logistic regression for typical clinical prediction problems.

Hold-Out Performance and Calibration

On the hold-out set the ranking matched cross-validation (Table 3, Figure 1). Logistic regression achieved the highest AUC (.823), accuracy (.750), and recall (.701), while the random forest had the lowest Brier score (.168), indicating slightly better calibration. Logistic regression missed 20 of the 67 diabetic patients and raised 28 false alarms among 125 non-diabetic patients. Because a positive screen would trigger a cheap confirmatory HbA1c test, whereas a missed case delays care, a threshold below 0.5 would be preferable in practice.

Table 3

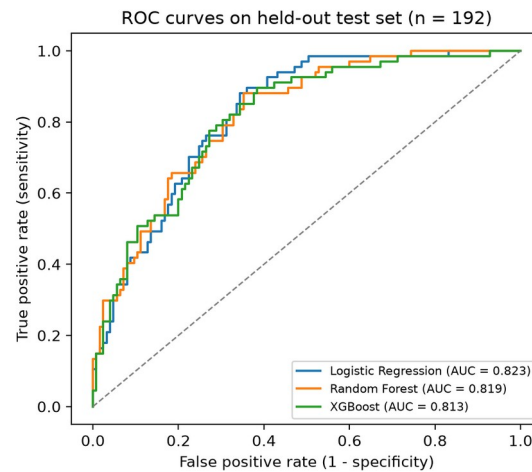
Hold-Out Test-Set Performance With Median Imputation (n = 192)

Model	ROC-AUC	Accuracy	Precision	Recall	F1	Brier
Logistic Regression	.823	.750	.627	.701	.662	.175
Random Forest	.819	.729	.597	.687	.639	.168
XGBoost	.813	.729	.603	.657	.629	.183

Note. Models trained on 576 records (75%) and evaluated once on 192 held-out records (67 positive). Threshold = 0.5.

Figure 1

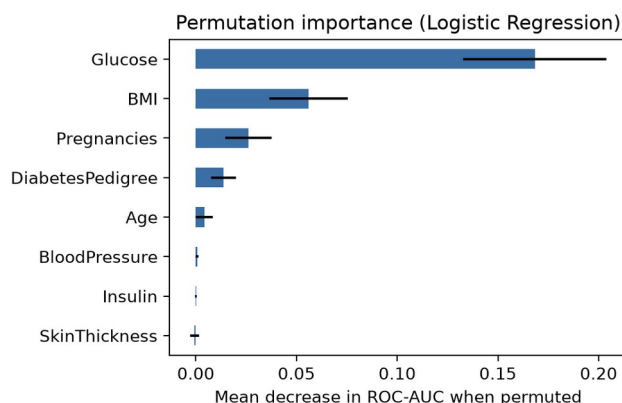
ROC Curves of the Three Classifiers on the Hold-Out Test Set



Note. Curves overlap almost entirely, illustrating the near-equivalence of the models. The dashed line represents chance performance.

Interpretability

Because logistic regression performed as well as the ensembles, its reasoning can be inspected directly, as Rudin (2019) recommends. Each standard-deviation increase in glucose multiplied the odds of diabetes by 3.20 and in BMI by 2.12; pregnancies (1.55), pedigree (1.31), and age (1.17) contributed less, and blood pressure, skinfold, and insulin had odds ratios near 1. Permutation importance (Figure 2) agreed: shuffling glucose reduced AUC by .168, BMI by .056, and every other feature by .026 or less. These patterns match clinical knowledge; a parsimonious model using glucose, BMI, age, and family history could suit settings where insulin or skinfold measurements are unavailable.

Figure 2*Permutation Importance of Predictors for the Logistic Regression Model*

Note. Bars show the mean decrease in hold-out ROC-AUC across 30 random permutations of each feature; error bars show one standard deviation.

Fairness Audit**Table 4***Subgroup Performance of Logistic Regression by Age on the Hold-Out Set*

Age group	<i>n</i>	Prevalence	Flagged positive	Sensitivity	FPR	AUC
Age < 30	109	.202	.211	.455	.149	.803
Age ≥ 30	83	.542	.627	.822	.395	.757

Note. FPR = false-positive rate. Flagged positive = proportion predicted diabetic at threshold 0.5. Random forest and XGBoost showed the same pattern (sensitivity .409 and .364 for under-30 patients).

Aggregate metrics concealed a substantial age disparity (Table 4). Logistic regression detected 82.2% of diabetic cases among patients aged 30 or older but only 45.5% among those under 30, and all three models showed the same pattern. Part of the gap reflects base rates (prevalence 20.2% vs. 54.2%). Yet AUC was slightly *higher* for the younger group (.803 vs. .757): the model ranks young patients well, but a single 0.5 threshold suits them poorly. Recalibration could narrow the gap (Hardt et al., 2016). With only 22 young diabetic patients the estimates are imprecise, but subgroup reporting should clearly be routine (Mehrabi et al., 2021; Rajkomar et al., 2018), since hidden disparities can do harm at scale (Obermeyer et al., 2019).

Practical and Ethical Implications

Practically, the Python stack makes rigorous prototyping cheap: all 600 model fits and figures came from one short open-source script, which matters for low-resource health systems. The scikit-learn Pipeline is a safeguard, not just a convenience, against the leakage Kapoor and Narayanan (2023) find is common. And because simple models perform as well as complex ones, organizations can favor transparent models clinicians can verify (Collins et al., 2015).

Ethically, the results caution against deploying a model on the strength of its overall AUC: under-detecting diabetes in younger adults would delay diagnosis for the group with the most years of potential complications ahead. Deployments must also protect patient privacy through de-identification and HIPAA or GDPR compliance. Finally, the data come from one Indigenous community, reused for decades with little benefit to it; responsible practice acknowledges this provenance and avoids claims about unrepresented populations.

Conclusion

Across twelve leakage-safe Python pipelines, recognizing impossible zeros as missing yielded small but consistent gains, while the choice of imputer and classifier mattered little. Interpretable logistic regression matched the ensembles and is the most defensible choice, but its sensitivity for patients under 30 was roughly half that for older patients, a weakness hidden by aggregate metrics.

The study contributes a replicable, fully open-source framework that treats rigor, calibration, and fairness as seriously as accuracy. Future work should validate it on larger, more diverse health records, test multiple imputation and group-aware recalibration, and test TensorFlow or Keras deep models. It could support primary-care and pharmacy screening once locally validated.

References

- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
<https://doi.org/10.1023/A:1010933404324>
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). ACM. <https://doi.org/10.1145/2939672.2939785>
- Christodoulou, E., Ma, J., Collins, G. S., Steyerberg, E. W., Verbakel, J. Y., & Van Calster, B. (2019). A systematic review shows no performance benefit of machine learning over logistic regression for clinical prediction models. *Journal of Clinical Epidemiology*, 110, 12–22. <https://doi.org/10.1016/j.jclinepi.2019.02.004>
- Collins, G. S., Reitsma, J. B., Altman, D. G., & Moons, K. G. M. (2015). Transparent reporting of a multivariable prediction model for individual prognosis or diagnosis (TRIPOD): The TRIPOD statement. *Annals of Internal Medicine*, 162(1), 55–63.
<https://doi.org/10.7326/M14-0697>
- Fisher, A., Rudin, C., & Dominici, F. (2019). All models are wrong, but many are useful: Learning a variable's importance by studying an entire class of prediction models simultaneously. *Journal of Machine Learning Research*, 20(177), 1–81.
- Hardt, M., Price, E., & Srebro, N. (2016). Equality of opportunity in supervised learning. In *Advances in Neural Information Processing Systems* (Vol. 29, pp. 3315–3323). Curran Associates.
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., ...

- Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362.
<https://doi.org/10.1038/s41586-020-2649-2>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- Kapoor, S., & Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9), Article 100804.
<https://doi.org/10.1016/j.patter.2023.100804>
- Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4), Article 15. <https://doi.org/10.1145/2382577.2382579>
- Kavakiotis, I., Tsave, O., Salifoglou, A., Maglaveras, N., Vlahavas, I., & Chouvarda, I. (2017). Machine learning and data mining methods in diabetes research. *Computational and Structural Biotechnology Journal*, 15, 104–116.
<https://doi.org/10.1016/j.csbj.2016.12.005>
- Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems* (Vol. 30, pp. 4765–4774). Curran Associates.
- McKinney, W. (2010). Data structures for statistical computing in Python. In S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (pp. 56–61).
<https://doi.org/10.25080/Majora-92bf1922-00a>
- Mehrabi, N., Morstatter, F., Saxena, N., Lerman, K., & Galstyan, A. (2021). A survey on bias and fairness in machine learning. *ACM Computing Surveys*, 54(6), Article 115.
<https://doi.org/10.1145/3457607>

- Nadeau, C., & Bengio, Y. (2003). Inference for the generalization error. *Machine Learning*, 52(3), 239–281. <https://doi.org/10.1023/A:1024068626366>
- Obermeyer, Z., Powers, B., Vogeli, C., & Mullainathan, S. (2019). Dissecting racial bias in an algorithm used to manage the health of populations. *Science*, 366(6464), 447–453. <https://doi.org/10.1126/science.aax2342>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Rajkomar, A., Dean, J., & Kohane, I. (2019). Machine learning in medicine. *New England Journal of Medicine*, 380(14), 1347–1358. <https://doi.org/10.1056/NEJMra1814259>
- Rajkomar, A., Hardt, M., Howell, M. D., Corrado, G., & Chin, M. H. (2018). Ensuring fairness in machine learning to advance health equity. *Annals of Internal Medicine*, 169(12), 866–872. <https://doi.org/10.7326/M18-1990>
- Rajpurkar, P., Chen, E., Banerjee, O., & Topol, E. J. (2022). AI in health and medicine. *Nature Medicine*, 28(1), 31–38. <https://doi.org/10.1038/s41591-021-01614-0>
- Rocklin, M. (2015). Dask: Parallel computation with blocked algorithms and task scheduling. In K. Huff & J. Bergstra (Eds.), *Proceedings of the 14th Python in Science Conference* (pp. 126–132). <https://doi.org/10.25080/Majora-7b98e3ed-013>
- Rudin, C. (2019). Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5), 206–215. <https://doi.org/10.1038/s42256-019-0048-x>

- Smith, J. W., Everhart, J. E., Dickson, W. C., Knowler, W. C., & Johannes, R. S. (1988). Using the ADAP learning algorithm to forecast the onset of diabetes mellitus. In *Proceedings of the Annual Symposium on Computer Application in Medical Care* (pp. 261–265). American Medical Informatics Association.
- Sterne, J. A. C., White, I. R., Carlin, J. B., Spratt, M., Royston, P., Kenward, M. G., Wood, A. M., & Carpenter, J. R. (2009). Multiple imputation for missing data in epidemiological and clinical research: Potential and pitfalls. *BMJ*, 338, Article b2393. <https://doi.org/10.1136/bmj.b2393>
- Sun, H., Saeedi, P., Karuranga, S., Pinkepank, M., Ogurtsova, K., Duncan, B. B., Stein, C., Basit, A., Chan, J. C. N., Mbanya, J. C., Pavkov, M. E., Ramachandaran, A., Wild, S. H., James, S., Herman, W. H., Zhang, P., Bommer, C., Kuo, S., Boyko, E. J., & Magliano, D. J. (2022). IDF Diabetes Atlas: Global, regional and country-level diabetes prevalence estimates for 2021 and projections for 2045. *Diabetes Research and Clinical Practice*, 183, Article 109119. <https://doi.org/10.1016/j.diabres.2021.109119>
- Topol, E. J. (2019). High-performance medicine: The convergence of human and artificial intelligence. *Nature Medicine*, 25(1), 44–56. <https://doi.org/10.1038/s41591-018-0300-7>
- Troyanskaya, O., Cantor, M., Sherlock, G., Brown, P., Hastie, T., Tibshirani, R., Botstein, D., & Altman, R. B. (2001). Missing value estimation methods for DNA microarrays. *Bioinformatics*, 17(6), 520–525. <https://doi.org/10.1093/bioinformatics/17.6.520>
- Van Buuren, S., & Groothuis-Oudshoorn, K. (2011). mice: Multivariate imputation by chained equations in R. *Journal of Statistical Software*, 45(3), 1–67. <https://doi.org/10.18637/jss.v045.i03>

- Van Calster, B., McLernon, D. J., van Smeden, M., Wynants, L., & Steyerberg, E. W. (2019). Calibration: The Achilles heel of predictive analytics. *BMC Medicine*, 17(1), Article 230. <https://doi.org/10.1186/s12916-019-1466-7>
- Waskom, M. L. (2021). seaborn: Statistical data visualization. *Journal of Open Source Software*, 6(60), Article 3021. <https://doi.org/10.21105/joss.03021>
- Zou, Q., Qu, K., Luo, Y., Yin, D., Ju, Y., & Tang, H. (2018). Predicting diabetes mellitus with machine learning techniques. *Frontiers in Genetics*, 9, Article 515. <https://doi.org/10.3389/fgene.2018.00515>

Appendix

Core Python Pipeline

The excerpt below shows the leakage-safe pipeline and the Dask-parallel evaluation loop.

The full, runnable script (analysis.py) reproduces every table and figure in this paper.

```
def preprocessor(kind):
    to_nan = FunctionTransformer(zeros_to_nan)          # 0 -> NaN for 5 columns
    if kind == "Raw (zeros kept)":
        return [("scale", StandardScaler())]
    if kind == "Median imputation":
        return [("nan", to_nan), ("imp", SimpleImputer(strategy="median")),
                ("scale", StandardScaler())]
    if kind == "KNN imputation":
        return [("nan", to_nan), ("scale", StandardScaler()),
                ("imp", KNNImputer(n_neighbors=5))]
    if kind == "Iterative (MICE-style)":
        return [("nan", to_nan), ("imp", IterativeImputer(max_iter=10)),
                ("scale", StandardScaler())]

cv = RepeatedStratifiedKFold(n_splits=10, n_repeats=5, random_state=42)

@dask.delayed
def evaluate(pre, model):
    pipe = Pipeline(preprocessor(pre) + [("clf", MODELS[model]())])
    return cross_validate(pipe, X, y, cv=cv, scoring=scoring)

tasks = [evaluate(p, m) for p in PREPS for m in MODELS]
results = dask.compute(*tasks, scheduler="threads")
```